

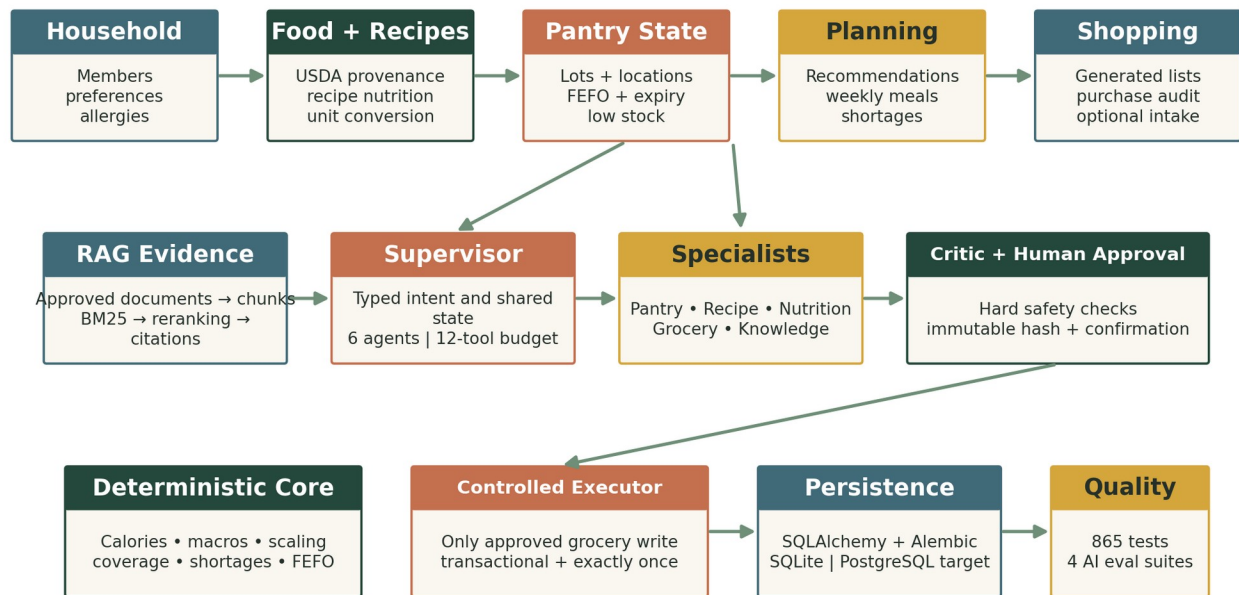
NOURISHNEST

NourishNest Technical Project Documentation

Production-oriented nutrition, pantry, RAG, multi-agent planning, and human-approved action

NourishNest End-to-End Intelligent Food Workflow

Household context becomes a grounded, safe, and human-approved grocery action



Streamlit UI • FastAPI • Pydantic • SQLAlchemy 2 • Alembic • BM25 • deterministic reranking • pytest

Fake provider supports a free offline demo; no model is trusted with domain arithmetic or direct writes

End-to-end product, AI, safety, and technology workflow

Submission edition • Implementation status through Phase 10C • September 2026

Document Purpose

This report documents the complete NourishNest build for technical review and weekly demonstration. It explains the user problem, system architecture, database evolution, application workflows, agentic AI design, evaluation framework, safety controls, verification evidence, current limitations, and production roadmap.

NourishNest has progressed from a household grocery concept into a tested full-stack and agentic application. Its central design decision is to separate language interpretation and evidence retrieval from deterministic calculations and transactional operations. A bounded supervisor coordinates five specialist agents and approved tools; offline RAG supplies exact citations; a deterministic critic and explicit human approval gate protect the only agent-assisted write: creation of a grocery list through an idempotent executor.

Contents

- 1 Executive Summary
- 2 Product Vision and User Problem
- 3 Functional Capabilities
- 4 System Architecture
- 5 Data Architecture and Migrations
- 6 Core Application Workflows
- 7 RAG and Agentic AI Architecture
- 8 Evaluation and Quality Engineering
- 9 Security and Reliability
- 10 User Experience
- 11 Phase History and Weekly Demonstration Guide
- 12 Operations and Configuration
- 13 Limitations and Roadmap
- Appendix A API Inventory
- Appendix B Demonstration Script

1 Executive Summary

NourishNest is an AI-assisted household food-management platform that connects household profiles, dietary needs, recipes, pantry inventory, meal planning, grocery generation, purchasing, nutrition calculations, grounded knowledge retrieval, and human-approved agent execution. The application uses FastAPI for typed HTTP services, Streamlit for the user experience, SQLAlchemy and Alembic for persistence, deterministic BM25 retrieval and reranking for offline RAG, and a bounded supervisor-specialist architecture for agent coordination.

Area	Current capability	Evidence
Application	Household, nutrition, recipes, pantry, grocery, planning, and assistant workflows	End-to-end browser smoke tests
Agentic AI	Supervisor, five specialists, typed state, bounded tools, critic, approval gate, and controlled executor	Phases 8 and 10B–10C
RAG	Offline ingestion, document revisions, deterministic chunking, BM25, reranking, and exact citations	36/36 retrieval cases
Evaluation	Software regression plus RAG, orchestration, safety, and approval evaluation suites	865 software tests passed
External data	USDA FoodData Central search, import, normalization, cache, and provenance	Provider tests and migration 0003
Safety	Allergen, medical, injection, tenant, tool-permission, proposal-hash, approval, and exactly-once controls	Guardrail and integrity tests
Deployment	Local Windows/uv workflow; SQLite development; PostgreSQL-compatible design	Production deployment remains pending

2 Product Vision and User Problem

Household food planning is fragmented. People maintain recipes in one place, groceries in another, and pantry information largely from memory. Nutrition targets rarely connect to actual ingredients, and food nearing expiration is discovered too late. NourishNest creates a continuous information flow from household needs to meal decisions and shopping actions.

The primary outcome is a practical answer to three everyday questions: What can we cook with what we have? Does it fit the household's needs? What exactly must we buy?

Design Principles

- Deterministic calculations are authoritative for calories, nutrients, quantities, conversions, inventory, and shortages.
- AI interprets language and coordinates approved tools; it does not invent operational facts.
- Household ownership is enforced at service and API boundaries.
- Potentially destructive or duplicate actions use confirmation, optimistic concurrency, idempotency, and database transactions.
- Incomplete or incompatible data produces explicit warnings rather than guessed conversions.
- The UI uses friendly names and hides technical identifiers under collapsed technical details.

3 Functional Capabilities

From Household Data to Actionable Planning



Figure 1. NourishNest capability layers

Domain	Implemented capability
Household	Create households and members; manage profile measurements, dietary preferences, and allergies; calculate saved-member nutrition.
Foods	Create and search normalized food records; import or refresh USDA FoodData Central items with provider-specific provenance.
Recipes	Browse system and household recipes; create, edit, delete, view ingredients and instructions, and calculate deterministic nutrition.
Pantry	Track named locations and multiple lots; expiration, FEFO consumption, transfers, adjustments, discards, low-stock rules, and audit events.
Grocery	Create lists and items; preview recipe requirements and pantry shortages; generate lists with lineage; record purchases and optional pantry intake.
Planning	Rank recipes against pantry coverage and expiring stock; build session-based weekly plans; compare nutrition and preview shortages.
Knowledge RAG	Ingest approved local documents; retrieve household-safe evidence using BM25 and deterministic reranking; return exact citations and injection warnings.
Multi-Agent Assistant	Coordinate Pantry, Recipe, Nutrition, Grocery, and Knowledge specialists through typed state, bounded tools, durable redacted traces, and deterministic synthesis.
Approval and Execution	Critique the preview, create an immutable proposal, require explicit human approval, and create a grocery list exactly once through a controlled executor.

4 System Architecture

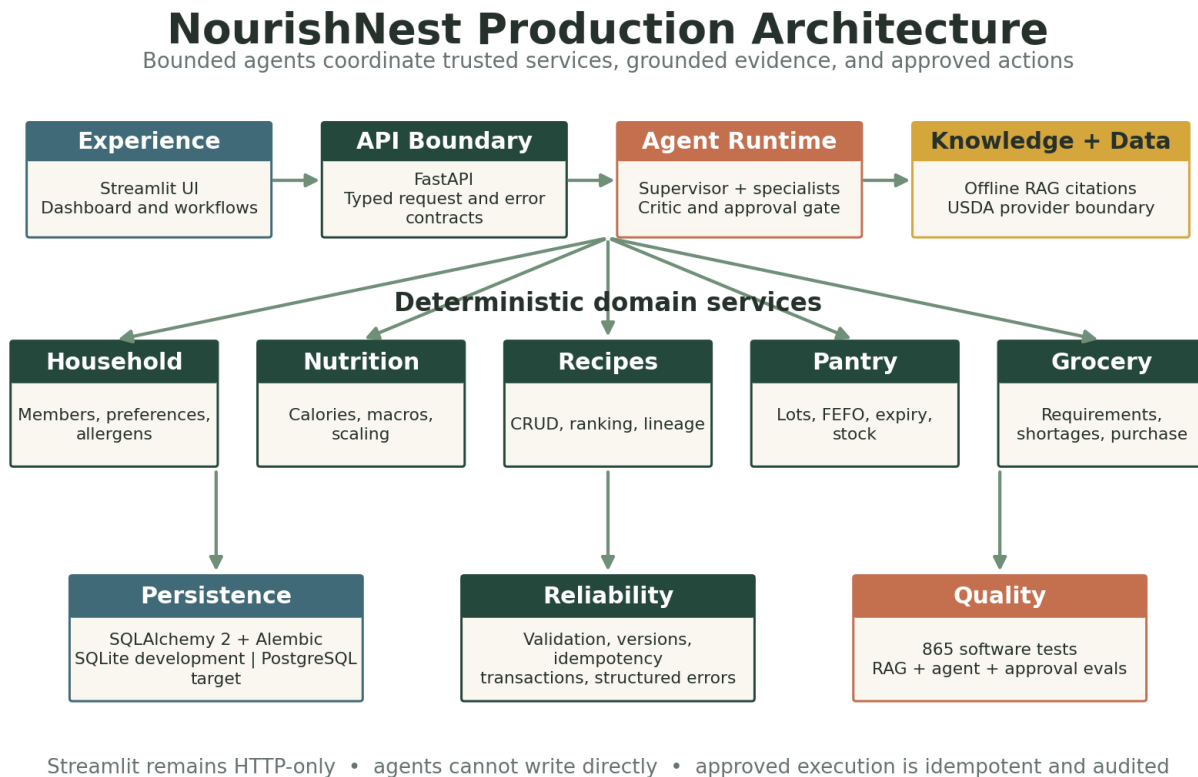


Figure 2. Layered application and AI architecture

4.1 Experience Layer

Streamlit provides the branded consumer interface. The UI calls FastAPI exclusively through a typed HTTP client; it does not import repositories or access the database. Navigation covers Dashboard, Household, Nutrition, Recipes, Pantry, Grocery Lists, Meal Planner, and AI Assistant. Session state retains active household, drafts, preview results, and retry keys where appropriate.

4.2 API and Service Layer

FastAPI exposes typed Pydantic contracts and a consistent error envelope containing code, message, and request ID. Services implement household isolation, validation, calculations, transaction boundaries, and orchestration. Repositories contain SQLAlchemy queries and provider-specific locking behavior.

4.3 Persistence Layer

SQLAlchemy 2 models use UUID primary keys, explicit timestamps, enumerated states, database constraints, relationship cascades, and NUMERIC fields for quantities and nutrition. Alembic manages incremental schema evolution. SQLite supports development and isolated tests; PostgreSQL is the intended production concurrency target.

4.4 Provider Boundaries

Food data, intent interpretation, retrieval, reranking, and agent tools are replaceable interfaces. USDA FoodData Central supplies structured external food data. The assistant provider supports disabled, deterministic fake, and optional local Ollama modes; the documented demo uses fake mode. Phase 10A uses offline lexical retrieval with no embedding model. Tests substitute fake providers and local fixtures, avoiding paid or live model calls.

4.5 Agent and Knowledge Plane

The agent plane sits above deterministic services rather than replacing them. A supervisor creates a bounded plan and coordinates Pantry, Recipe, Nutrition, Grocery, and Knowledge specialists. The Knowledge Agent calls the Phase 10A retrieval service; other agents call typed domain tools. Durable traces store safe summaries, statuses, timing, and tool names without full prompts, document bodies, secrets, or chain-of-thought.

4.6 Human-Controlled Action Plane

A deterministic critic checks household scope, allergens, dietary constraints, quantities, lineage, citations, budgets, and source-run integrity. An immutable proposal captures the planned grocery action and a SHA-256 payload hash. Proposal creation and approval perform no domain write. Only a separate, explicitly confirmed executor may create a grocery list, and database idempotency protects exactly-once behavior.

5 Data Architecture and Migrations

Revision	Purpose	Primary schema contribution
0001	Household foundation	Households, members, dietary preferences, allergies
0002	Food and recipe foundation	Foods, allergens, tags, recipes, ingredients, instructions
0003	USDA provenance	Provider identity, external identifier, attribution, retrieval metadata
0004	Pantry	Locations, lots, stock rules, append-only transaction history
0005	Grocery foundation	Grocery lists and versioned grocery-list items
0006	Grocery generation	Generation runs, recipe lineage,

Revision	Purpose	Primary schema contribution
		generation references
0007	Purchase events	Purchase audit and optional pantry-intake linkage
0008	Member concurrency	Optimistic member version column and scoped mutation contract
0009	Recipe concurrency	Recipe versions and normalized creation idempotency keys
0010	RAG knowledge foundation	Auditable documents and revisions, deterministic chunks, hashes, visibility and retrieval metadata
0011	Multi-agent trace	Redacted agent runs and steps, statuses, tool summaries, timing and household ownership
0012	Approval and execution	Immutable action proposals, approval events, executions, hashes, versions and idempotency

5.1 Core Entity Relationships

- A household owns members, pantry locations, pantry lots, stock rules, grocery lists, planning context, and household recipes.
- Foods are shared reference records and may carry manual, development-fixtured, or external-provider provenance.
- Recipes contain ordered ingredients and instructions; ingredients reference foods and drive nutrition and grocery requirements.
- Pantry items represent lots, preserving location, quantity, canonical unit, expiration, status, and version.
- Grocery generation runs link selected recipes and generated items to auditable source records.
- Purchase events prevent duplicate intake and may atomically create a pantry lot and restock transaction.
- Knowledge documents and chunks preserve global or household visibility, stable content hashes, ordered chunk positions, source metadata, and auditable revisions.
- Agent runs own redacted execution steps; approval proposals reference eligible runs and retain append-only decision and execution evidence.

5.2 Numeric Integrity

Quantities and nutrient values use Decimal in application code and NUMERIC(18,6) where precision is operationally important. Mass, volume, and count remain separate dimensions. The application converts compatible units into canonical grams, milliliters, or items and refuses to infer density.

6 Core Application Workflows

6.1 Nutrition

Member inputs are validated, converted into a shared nutrition profile, and processed by deterministic formulas. Recipe nutrition scales each ingredient through dimension-aware unit conversion, aggregates calories and macros, divides totals by servings, propagates allergens, and reports incomplete components as warnings.

6.2 Pantry Inventory

Pantry intake creates versioned lots. Consumption uses first-expiring-first-out selection across compatible active lots. Adjustments, transfers, discards, and restocks are atomic and append audit transactions. PostgreSQL queries use row locks where supported; optimistic versions reject stale writes. Idempotency keys prevent duplicated mutations.

6.3 Recipe Requirements and Shortages

A recipe selection is scaled to desired servings. Ingredient quantities are converted and aggregated by food and canonical dimension. Shortage preview subtracts active compatible pantry availability from requirements. Expired inventory is excluded, incompatible dimensions remain separate, and uncertain conversions become structured warnings.

6.4 Grocery Generation and Purchase

Generation atomically writes a run, positive-shortage items, recipe lineage, and a list-version update. Request fingerprints canonicalize recipe ordering and Decimal servings for idempotent replay. Purchases update item progress, completion state, list version, and purchase audit; optional pantry intake creates stock exactly once.

6.5 Recipe Recommendation

Recipe ranking is deterministic: 80 percent of the score reflects average quantity coverage and 20 percent reflects average coverage by expiring stock. Coverage is capped at one per required food and unit. Unsupported conversions score zero and emit warnings. Ties resolve by coverage, recipe name, and recipe ID.

7 RAG and Agentic AI Architecture

Offline Grounded Retrieval Pipeline

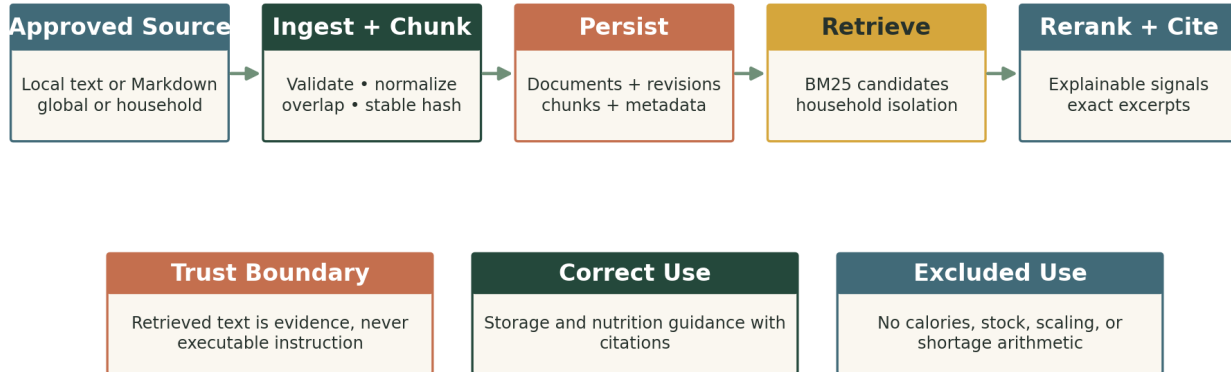


Figure 3. Offline retrieval, reranking, and citation pipeline

7.1 Phase 10A Offline RAG

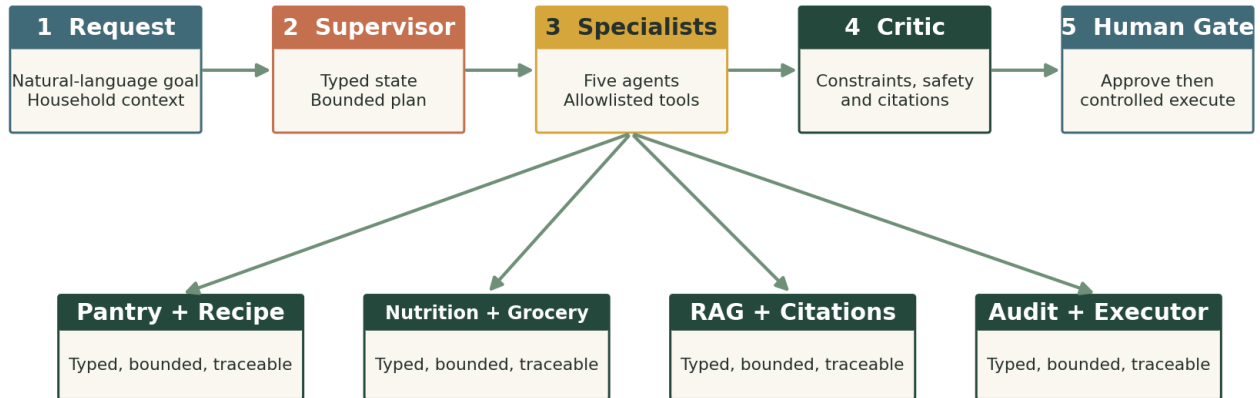
Phase 10A introduced an offline, auditable retrieval layer for approved nutrition, food-safety, recipe, and household documents. Ingestion validates an approved local path, normalizes content, calculates a content hash, and applies deterministic structure-aware chunking with stable overlap. Revisions and chunks are persisted with source metadata and global or household visibility.

Retrieval uses BM25-style lexical scoring to select candidates. A separate deterministic reranker combines lexical score, normalized term overlap, title and heading matches, phrase matches, and configured source priority. Results contain exact stored excerpts and structured citations. Retrieved text is treated as untrusted evidence; common prompt-injection language is flagged and never executed.

RAG is appropriate for	RAG is not authoritative for
Explanatory nutrition and food-safety guidance with citations	Calories, macronutrients, or serving calculations
Approved household documents and user recipe notes	Pantry quantities, expiration status, or FEFO selection
Storage, preparation, and educational evidence	Unit conversion, recipe scaling, or grocery shortages
Evidence supplied to the Knowledge Agent	Database mutations or tool permissions

Safe Agentic Meal Planning Flow

Interpret, delegate, retrieve, calculate, critique, approve, execute, and audit



Agents are read-only; only the deterministic executor may create an approved grocery list exactly once

Figure 4. Bounded multi-agent and human-approval workflow

7.2 Phase 10B Supervisor and Specialists

A supervisor translates a supported household goal into typed shared state, creates a deterministic execution plan, and coordinates five specialists: Pantry, Recipe, Nutrition, Grocery, and Knowledge. Independent read-only agents may run concurrently; dependent work waits for recipe selection. The complete recorded workflow is limited to six agent executions and twelve tool calls, with timeouts, stable ordering, cancellation, and explicit partial-failure behavior.

Agent	Responsibility	Representative tools
Supervisor	Validate intent, select agents, enforce budgets, merge results	No domain arithmetic
Pantry	Read availability, expiring lots, and low stock	Pantry summary and expiry queries
Recipe	Select accessible recipes under constraints	Recipe listing and deterministic recommendation
Nutrition	Calculate recipe/member and weekly totals	Existing nutrition services
Grocery	Calculate requirements and pantry shortages	Existing preview services

Agent	Responsibility	Representative tools
Knowledge	Retrieve approved explanatory evidence	Phase 10A retrieval and citations

The tool registry is allowlisted and typed. It rejects unknown tools, malformed arguments, unauthorized agent-tool combinations, write tools, and budget overruns. Traces record safe input/output summaries, tool names, statuses, warnings, timing, and failures; they exclude full private prompts, document bodies, secrets, and chain-of-thought.

7.3 Phase 10C Critic and Human Approval

The Critic Agent is deterministic and read-only. It verifies household scope, source-run status, budgets, allergens, dietary constraints, recipe access, quantities, grocery lineage, citation integrity, injection warnings, and proposal allowlisting. It returns pass, pass with warnings, or block with concise structured reasons.

An eligible result becomes an immutable grocery-action proposal. Canonical UUIDs, Decimal strings, ordered recipes and foods, versions, timestamps, lineage, and calculation versions are hashed with SHA-256. The human must explicitly approve the matching hash and then separately request execution. Only the controlled executor can create the grocery list; replay of the identical request returns the saved outcome without duplication.

Stage	Domain write?	Primary control
Multi-agent preview	No	Read-only allowlisted tools
Critic review	No	Hard blockers and structured warnings
Proposal creation	No	Immutable canonical payload and hash
Human approval	No	Confirmation, expiry, and optimistic version
Controlled execution	Grocery list only	Allowlist, transaction, idempotency, exactly once
Purchase or pantry mutation	Not permitted	Outside the Phase 10C executor

7.4 Deterministic and AI Boundary

AI and agent layer	Deterministic application layer
Interpret supported natural-language constraints	Household ownership and record access
Plan bounded specialist participation	Calories, macros, unit conversion, recipe scaling
Retrieve and cite explanatory evidence	Pantry availability, FEFO, expiration, shortages
Summarize structured decisions	Transactions, versions, hashes, idempotency, writes
Ask for clarification or refuse unsafe scope	Critic checks and controlled execution

7.5 Provider Modes and Guardrails

Mode	Purpose	External model	Cost
disabled	Safe default	None	\$0
fake	Deterministic tests and offline demonstrations	None	\$0
ollama	Optional local interpretation adapter	Local process	\$0 API cost

- Allergy conflicts, cross-household access, invalid quantities, missing lineage, unsafe medical requests, and tampered proposal hashes are hard blockers.
- Prompt injection in a user request or retrieved document cannot expand tool permissions or change system behavior.
- Diagnosis, treatment, dangerous calorie targets, guaranteed outcomes, and requests for hidden prompts or chain-of-thought are refused.
- Agents cannot delegate recursively, create new agents, call arbitrary functions, or write domain records.
- Nutrition remains informational and does not replace medical or dietetic advice.

8 Evaluation and Quality Engineering

8.1 Evaluation Strategy

NourishNest treats evaluations as release evidence rather than a demonstration afterthought. Versioned golden datasets run with fake providers, local fixtures, isolated databases, stable expected outputs, explicit thresholds, and nonzero failure exit codes. They measure retrieval, orchestration, safety, approval, and deterministic outcomes without a paid model.

Suite	Cases	Primary coverage	Reported result
Meal-planning assistant	49	Intent, tools, allergens, clarification, refusal, grounding	49/49 passed
Knowledge retrieval v1	36	Hit@k, MRR, citations, isolation, filters, no-answer	36/36; nine metrics 1.00
Multi-agent planning v1	54	Agent/tool selection, budgets, ordering, constraints, citations	54/54; accuracy metrics 1.00
Approval execution v1	58	Critic, hashes, transitions, idempotency, rollback, exactly once	58/58; required metrics 1.00

8.2 What the Metrics Establish

Metric family	What it verifies
Retrieval quality	Expected evidence appears in top ranks and citations reference exact stored chunks
Tenant isolation	Global and own-household evidence is visible; another household's evidence is not
Agent selection	The supervisor activates only specialists required by the typed request
Tool authorization	Each specialist uses only allowlisted read-only tools within the shared budget
Constraint safety	Allergens are hard exclusions and dietary, time, serving, and

Metric family	What it verifies
	medical boundaries hold
Approval integrity	Tampering, stale versions, invalid transitions, expiry, and conflicting replays fail safely
Exactly-once action	Identical execution replay does not duplicate a grocery list or its items
Mutation boundary	Preview, critic, proposal, and approval do not change domain tables

8.3 Verification Evidence

Checkpoint	Result
Phase 8A assistant service	453 tests passed; 49/49 AI evaluation cases passed
Phase 9A visual redesign	550 tests passed; all checks passed
Phase 10A RAG	702 tests; 98 focused; 36/36 retrieval cases
Phase 10B multi-agent	787 tests; 202 focused; 54/54 orchestration cases; p95 139.42 ms
Phase 10C approval execution	865 final full-suite tests passed; 58/58 approval cases
Static analysis	Ruff passed
Patch hygiene	git diff --check passed

These results establish deterministic contract behavior, safety boundaries, retrieval quality against frozen local datasets, and transaction integrity. They do not measure an untested real language model's open-ended reasoning quality. Fake mode is intentionally repeatable and cost-free; Ollama remains optional and was not deployed for the documented workflow.

9 Security and Reliability

Control	Implementation
Tenant isolation	Household-scoped routes and ownership validation; inaccessible records return structured 404 responses
Input safety	Pydantic validation, strict enums, Decimal contracts, typed provider output
Concurrency	SQLAlchemy optimistic version columns and stale-write conflicts
Idempotency	Database-backed keys and canonical request fingerprints for mutation replay protection
Atomicity	Service-level transactions roll back inventory, grocery, lineage, and audit changes together
Auditability	Pantry/purchase events, generation lineage, provenance, redacted agent traces, approval events, executions, and request IDs
Provider resilience	Timeouts, bounded retries for safe reads, error classification, TTL caching, secret suppression
AI safety	Typed state, specialist permissions, budgets, timeouts, prompt-injection warnings, critic blockers, and no direct agent writes
Human control	Immutable proposal hashes, explicit confirmation, separate execution, allowlisted action, and exactly-once replay

Current Security Gap

Authentication and end-user authorization are not yet implemented. Household scoping protects records within the current API contract, but there is no verified identity binding a caller to permitted households. This is the most important gap before public production deployment.

10 User Experience

The Streamlit experience was redesigned around a warm sage, cream, and terracotta brand. Navigation groups planning, shopping, management, and profile tasks. Friendly names replace UUID-heavy selectors; technical identifiers and request IDs appear only in collapsed details. Local WebP imagery, original SVG assets, responsive cards, professional empty states, and confirmation-gated destructive actions support a consumer-facing presentation.

Page	Primary experience
Dashboard	Household overview, expiring food, recommendations, weekly preview, quick actions
Household	Member cards, profile measurements, dietary preferences, allergies
Nutrition	Named member selection and server-calculated energy/macronutrient targets
Recipes	Visual browsing, detail, nutrition, household recipe editing
Pantry	Locations, lots, expiry, consume, adjust, transfer, discard, low stock
Grocery Lists	Requirement/shortage preview, generation, shopping progress, purchase intake
Meal Planner	Recommendations, seven-day slots, weekly nutrition, grocery preview
AI Assistant	Conversational request, multi-agent trace, RAG citations, critic review, human approval, and controlled grocery action

11 Phase History and Weekly Demonstration Guide

Phase	Theme	Demonstrable outcome
1	Data foundation	Households, members, preferences, allergies, FastAPI persistence
2	Foods and recipes	Recipe model, deterministic nutrition, unit conversion, seed fixtures
3	External food data	USDA provider, cache, retries, provenance, import and refresh
4	Pantry	Locations, lots, FEFO, expiration, low stock, integrity audit
5	Grocery lifecycle	CRUD, requirements, shortage, generation, lineage, purchase and pantry intake
6	Application UI	Streamlit shell and complete household, recipe, pantry, and grocery workflows
7	Smart planning	Deterministic recommendations and weekly meal-planning experience
8	Agentic AI	Single orchestrator, guardrails, evals, Assistant UI, optional Ollama adapter
9	Product polish	Brand system, responsive layouts, relevant local imagery, visual QA
10A	Offline RAG	Document revisions, chunking, BM25 retrieval, reranking, exact citations, retrieval evals
10B	Multi-agent orchestration	Supervisor, five specialists, typed state, bounded tools, durable redacted traces
10C	Human-approved execution	Critic, immutable proposals, approval lifecycle, controlled exactly-once grocery creation

Suggested Weekly Demo Narrative

Demo	Focus	What to show
Week 1	Problem and foundation	Show household creation, member preferences, architecture, and persistence.
Week 2	Nutrition and recipes	Create/search food, show recipe ingredients, deterministic conversions, and nutrition.
Week 3	Pantry intelligence	Add two lots, demonstrate expiry status, FEFO consumption, transfer, and low stock.
Week 4	Grocery automation	Scale recipes, preview requirements and shortages, generate a list, purchase with intake.
Week 5	Planning	Rank recipes by pantry and expiring-stock coverage; build a seven-day plan.
Week 6	Agent and evaluation	Submit a natural-language goal; show typed intent, tool trace, guardrail, and 49-case report.
Week 7	Product quality	Show branded responsive UI, test evidence, risks, and production roadmap.
Week 8	RAG	Ingest an approved document; show chunks, BM25 ranking, reranking, exact citation, and injection warning.
Week 9	Multi-agent system	Show supervisor, five specialists, permissions, parallel reads, shared state, and traces.
Week 10	Human-approved action	Critique a plan, approve its immutable proposal, execute once, replay safely, and inspect audit evidence.

12 Operations and Configuration

12.1 Local Startup

Run FastAPI and Streamlit from the repository root in separate PowerShell terminals.

```
# Terminal 1
$env:UV_LINK_MODE = "copy"
$env:APP_AI_PROVIDER = "fake"
uv run python -m uvicorn nourish_nest.api:app --host 127.0.0.1 --port 8000 --reload

# Terminal 2
$env:UV_LINK_MODE = "copy"
$env:APP_API_BASE_URL = "http://127.0.0.1:8000"
uv run python -m streamlit run streamlit_app.py --server.port 8501
```

12.2 Verification Commands

```
uv sync --extra dev
uv run alembic upgrade head
uv run python -m pytest
uv run ruff check .
git diff --check
uv run python -m nourish_nest.evals.meal_planning
uv run python -m nourish_nest.evals.knowledge_retrieval
uv run python -m nourish_nest.evals.multi_agent_meal_planning
uv run python -m nourish_nest.evals.approval_execution
```

12.3 Key Environment Configuration

Variable	Purpose	Typical local value
APP_DATABASE_URL	Database connection	SQLite default or PostgreSQL URL
APP_API_BASE_URL	Streamlit to FastAPI address	http://127.0.0.1:8000
APP_FOOD_DATA_PROVIDER	External food source	disabled or usda
USDA_API_KEY	FoodData Central credential	Secret environment value
APP_AI_PROVIDER	Assistant interpretation mode	disabled, fake, or ollama
APP_AI_MODEL	Optional local model name	Empty unless Ollama is used
APP_OLLAMA_BASE_URL	Optional local inference endpoint	http://127.0.0.1:11434
APP_AI_TIMEOUT_SECONDS	Model/provider timeout	Configured bounded value

13 Limitations and Roadmap

13.1 Current Limitations

- Authentication, identity federation, role-based household membership, and authorization are not implemented.
- SQLite is appropriate for development but does not provide PostgreSQL-equivalent row-level locking or Decimal storage behavior.
- Meal-plan drafts and conversational presentation state remain session-local, although agent runs, steps, proposals, approval events, and executions are durable audit records.
- Pantry availability and recipe recommendations are point-in-time estimates without reservation of stock.
- The fake provider demonstrates orchestration, not open-ended language-model quality; Ollama was implemented but not deployed.
- Allergen matching uses normalized stored names without clinical synonym inference; nutrition output is informational, not medical advice.
- Phase 10A retrieval is lexical and scans an in-memory candidate corpus; semantic embeddings, a vector database, and distributed retrieval are not implemented.
- Prompt-injection detection is heuristic. Retrieved content remains untrusted and cannot change tool permissions, but authoritative source governance still requires an operational process.
- Human approval is household-scoped but not identity-verifiable because authentication is not implemented.
- Existing generation rules limit one generation per grocery list; refund and purchase-reversal workflows are deferred.

13.2 Recommended Production Roadmap

Priority	Workstream	Outcome
P0	Identity and authorization	Managed authentication, user-to-household roles, token validation, endpoint policy tests
P0	Production data	PostgreSQL deployment, migration pipeline, backups, restore test, connection pooling
P1	Delivery	Container hardening, CI/CD, secret management, environment promotion, rollback
P1	Observability	Structured logs, readiness, metrics, traces, alerting, audit review

Priority	Workstream	Outcome
P1	Durable planning	Persisted meal plans and conversation summaries with privacy controls
P2	Retrieval scale	PostgreSQL full-text or vector retrieval, semantic evaluation, source governance, freshness, and re-indexing
P2	Model validation	Evaluate an approved provider against frozen datasets, cost, latency, privacy, and safety release gates
P3	Controlled action expansion	Reservations, reversals, and additional actions only with dedicated critic, approval, and audit policies

Appendix A API Inventory

Domain	Representative route	Purpose
Health	GET /health	Service status and request ID
Households	GET/POST /v1/households	Discover and create households
Members	GET/POST /v1/households/{household_id}/members	Member collection
Members	GET/PUT/DELETE .../members/{member_id}	Scoped member operations and versions
Nutrition	POST /nutrition/calculate	Standalone deterministic target calculation
Foods	POST /v1/foods; GET /v1/foods/search	Stored-food creation and search
USDA	GET /v1/providers/usda/search and /foods/{fdc_id}	Provider search and detail
USDA	POST /v1/foods/import/usda/{fdc_id}	Import and refresh external food

Domain	Representative route	Purpose
Recipes	CRUD /v1/households/{household_id}/recipes	Household/system recipe workflows
Recipes	GET .../recipes/{recipe_id}/nutrition	Deterministic recipe nutrition
Pantry	.../pantry/locations and .../pantry/items	Locations and lot inventory
Pantry	.../consume, adjust, transfer, discard	Atomic stock mutations
Pantry	.../expiring, expired, low-stock, summary	Inventory intelligence
Grocery	CRUD .../grocery-lists and items	List and manual-item lifecycle
Requirements	POST .../grocery-requirements/preview	Scaled recipe needs
Shortage	POST .../grocery-requirements/shortage-preview	Pantry-aware purchase needs
Generation	POST .../grocery-lists/{list_id}/generations	Idempotent generated list
Purchase	POST .../items/{item_id}/purchase	Audit and optional pantry intake
Planning	POST .../recipe-recommendations	Deterministic ranked recipes
Assistant	POST .../assistant/meal-plan-preview	Read-only agentic preview
Knowledge	POST .../knowledge/retrieve	Household-safe BM25 retrieval, reranking, and citations
Multi-agent	POST .../assistant/multi-agent-meal-plan-preview	Bounded supervisor and specialist preview
Controlled action	POST .../runs/{run_id}/proposals; .../proposals/{id}/approve reject cancel execute	Critic-eligible proposal, human decision, and exactly-once grocery action

Appendix B Demonstration Script

Step	Action	Talking point
1	Open Dashboard	Introduce the household food-management problem and show the professional shell.
2	Choose Household	Show named household and member context, preferences, and allergens.
3	Inspect Pantry	Show locations, two lots, expiry status, low stock, and FEFO behavior.
4	Browse Recipes	Open a recipe, ingredients, instructions, nutrition, and pantry-match rationale.
5	Plan Week	Add recommended recipes to days and display weekly nutrition and shortages.
6	Retrieve Evidence	Ask a storage or nutrition question; show BM25 result, rerank signals, citation, and source isolation.
7	Ask Multi-Agent Assistant	Request five vegetarian dinners, peanut exclusion, expiring-stock priority, time limit, and shortages.
8	Explain Orchestration	Show supervisor, five specialists, typed state, tool permissions, budgets, citations, and redacted trace.
9	Review Critic	Show verified constraints, warnings, and the hard-block behavior for an allergy conflict.
10	Approve	Create the immutable proposal, verify its hash, and explicitly approve without changing grocery data.
11	Execute Once	Create the grocery list through the controlled executor and replay without duplication.
12	Show Evaluation	Display RAG, orchestration, approval,

Step	Action	Talking point
		and 865-test evidence while explaining fake-mode scope.
13	Close	Summarize grounded evidence, deterministic calculations, human control, limitations, and roadmap.

Suggested Opening

NourishNest is a production-oriented household nutrition and food-management application. It connects member needs, recipes, pantry inventory, meal planning, grounded knowledge, and grocery shopping. The key technical idea is separation of responsibilities: agents understand and coordinate, RAG supplies cited evidence, deterministic services calculate, a critic verifies, the human approves, and a controlled executor performs the single permitted action exactly once.

Suggested Closing

The project demonstrates more than a chat interface. It combines a production-oriented domain model, deterministic household calculations, offline citation-backed RAG, bounded multi-agent orchestration, durable redacted traces, critic validation, human approval, exactly-once execution, versioned evaluations, and a polished user experience. The next production priorities are authentication, live PostgreSQL validation, CI/CD, secrets, monitoring, source governance, and real-provider evaluation.